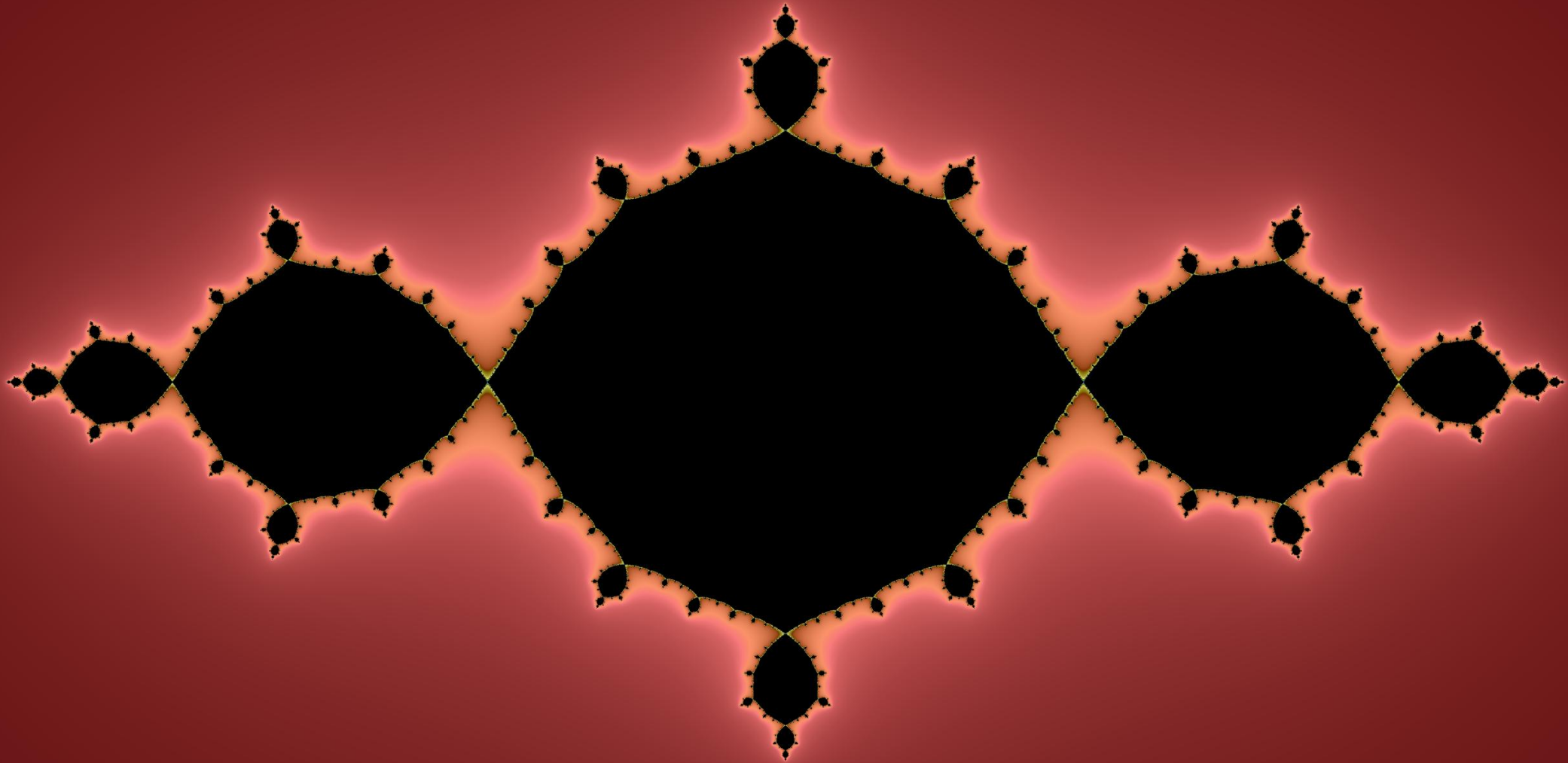


Fractal Software Design – Software Portability and Self Similarity (HAUD)

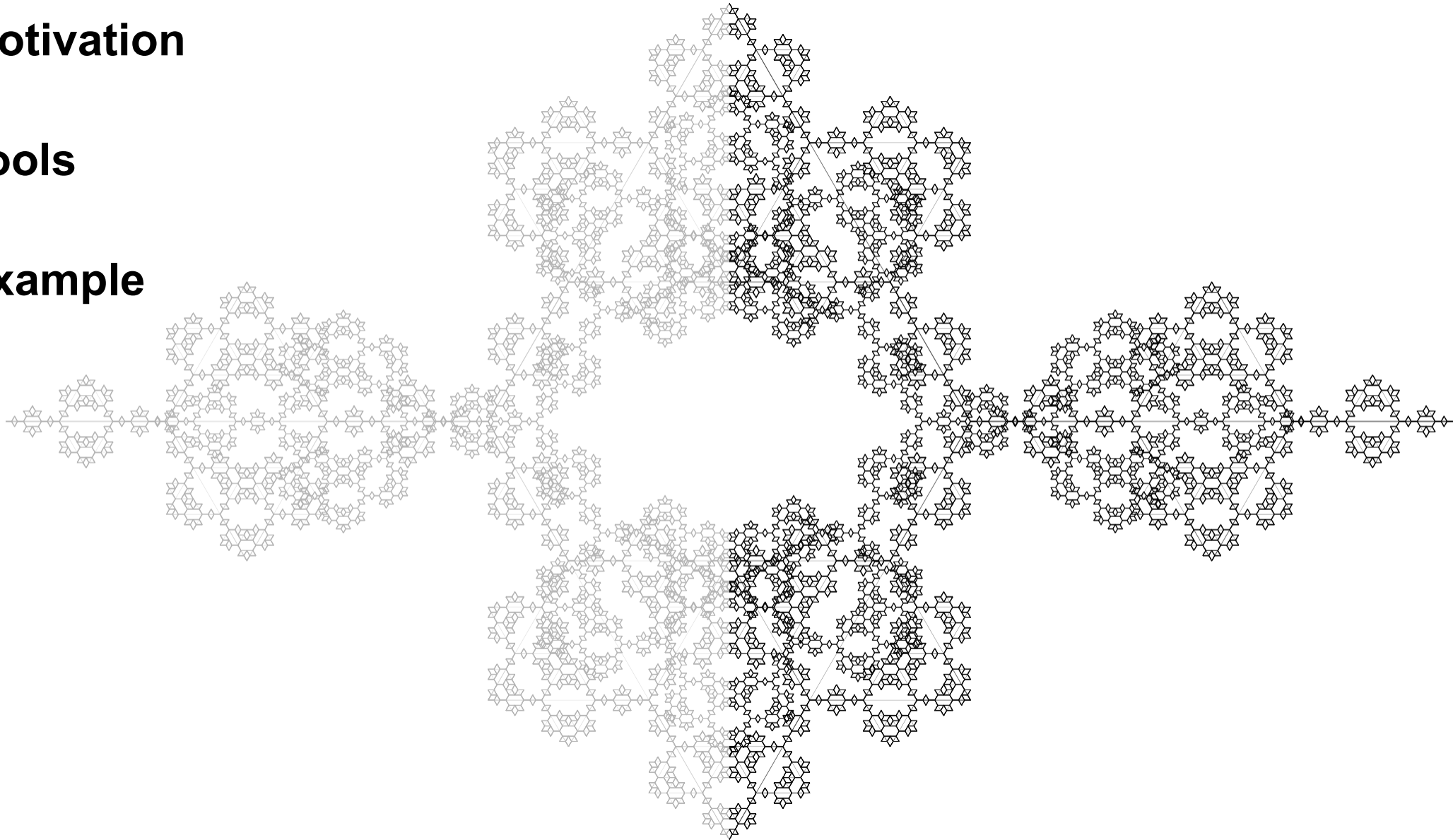
BECKHOFF



1. Motivation

2. Tools

3. Example



1. Motivation

- **Software portability**
 - Top level business logic shall not suffer from changes in lower abstraction levels.
- **Rigid hardware abstraction**
 - Unified software mechanics to shield business logic from hardware variance.
- **No God Objects**
 - Edge cases shall not disrupt abstraction.
- **Same cycle responsiveness**
 - Eliminate traditional, monolithic state machine design flaws.

1. Motivation

- **Software portability**
 - **Separation of business logic and motion logic enables portability.**
- **Changes in motion layer:**
 - Adding new features does not break the code.
 - Swapping a hardware controller does not break your business logic.
 - Introducing a new motion kernel does not break the code.

1. Motivation

- **Hardware Abstraction Layer**
 - **Strict interfaces enforce behavioural contract.**

Rigid = Enforced Contract = Safety

- **Rigid Interfaces ensure:**
 - Business logic never couples to hardware.
 - Hardware changes do not affect business logic.
 - Different hardware can be swapped transparently.

1. Motivation

- **NO God Objects**
 - **Work local, ship global.**
 - **Composition over inheritance.**
- **Composition ensures:**
 - Specialized logic can be reused.
 - Scales easily with future development.
- **Know Thyself:**
 - Know your job and do it well

1. Motivation

- **Same Cycle Responsiveness and Cycle Optimal Programming**
 - **Break the cycle.**
 - **Find a reusable structure to break safely.**
 - **Logic progression:**
 - Do not waste time to switch decision trees.
 - Ensure observability.
 - Keep decisions simple

1. Motivation

– **WHY?**

- **Moving at 10 m/s requires cycle optimal programming.**
- **Synchronizing place and time with software that is fast and safe.**
- **The machine must observe itself and tell the user about it.**
- **Communication agnosticism is a requirement for distributed motion applications.**
- **Top application layer must be oblivious to the hardware executing the commands.**

2. Tools

▪ SOLID DESIGN

- **S** — Single Responsibility Principle
 - a class should have one, and only one, reason to change.
- **O** — Open/Closed Principle
 - software entities should be open for extension but closed for modification.
- **L** — Liskov Substitution Principle
 - Objects of a superclass shall be replaceable with objects of its subclasses without breaking the application.
- **I** — Interface Segregation Principle
 - users should not be forced to depend upon interfaces that they do not use.
 - a software entity only sees the methods it actually needs to do the work.
- **D** — Dependency Inversion Principle
 - Depend upon abstractions, not concretions.

2. Tools

▪ ACID-inspired principles for cyclic RTS

Borrowed from database transaction properties and adapted to deterministic control logic.

- **A** — Atomicity
 - Within a cyclic real-time task, a bounded local logic step can be treated as one deterministic, observable operation.
- **C** — Consistency
 - A transaction must bring the system from one valid state to another valid state.
 - The required handshake sequences perform a consistent execution. The behaviour is fully testable.
- **I** — Isolation
 - Concurrent transactions should not interfere with each other.
 - The architecture provides structural isolation by design.
- **D** — Durability
 - Once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.
 - The loss of state or information must lead to a coordinated recovery.

2. Tools

▪ Clarifications

- The **SOLID** tools are directly applicable in TwinCAT OOP.
- The decision how deep inheritance runs is influenced by the task at hand.
- In my world, a flat inheritance tree has certain advantages regarding variations and the taming of complexity.
 - Since there is no “one size fits all”
 - → the depth of inheritance is up for consideration
- Base classes convey the intent of custom extensions that solve an adjacent problem or substitute an existing implementation without changing the code.
- Interfaces are used as horizontal connector for classes that might share common properties.

2. Tools

▪ Clarifications

- The **ACID** tools require some explanations.
 - Transactional and relational principles share properties that are also applicable to PLC programming.
- **Atomicity (Logic)**: The cyclic real-time task provides a deterministic execution boundary for bounded local logic. Within that boundary, internal decisions can be executed as one observable logical step.
- We now only have to compress the traditional state machine changes (that consume a PLC cycle) into one observable sequence that executes without cycle change. By employing such sequential state execution, the logic needs only to wait for feedback when it is required.
- Further is it helpful to eradicate the looping of arrays and classic buffers
 - Preallocated, bounded linked structure can avoid array shifting. „GetHead“ and „AddTail“ provide $O(1)$ FIFO operations.
 - Preallocation prevents runtime memory allocation and fragmentation risk.

2. Tools

▪ Clarifications

- The **ACID** tools require some explanations.
 - A lot of similar problems are coded in vastly different ways, mostly caused by crunching time during FAE.
- **Consistency (architecture):** The facade pattern as helper
 - If everything works the same, reading the code is made easier.
- Transactional handshakes have two trade offs
 - Easy to read
 - Easy to maintain
- A **consistent** use of the facades for handshakes:
 - preserves defined machine invariants
 - provides one observable outcome for every command:
 - **accepted, rejected, executing, completed, aborted or faulted.**

2. Tools

▪ Clarifications

- The **ACID** tools require some explanations.
 - Preventing unintended or unauthorized modification of validated cyclic logic and state is essential.
- **Isolation (architecture)**: Again; the facade pattern as helper, and the horizontal interface pointer as guard.
 - If everything works an addition or deletion of functionality somewhere else must not break the architecture.
- Isolation by design is the best shield
 - Semaphores in a cyclic RTS system add shared synchronization state, timing risks, and maintenance obligations. Prefer explicit ownership and cyclic handover wherever possible.
- Each mutable Ctrl / State facade has one defined cyclic owner. Other tasks, mappings, ADS clients, and application layers submit requests through the facade and do not directly alter internal cyclic state.
- Architectural isolation prevents unwanted dependencies
 - ownership rules prevent conflicting writers.

2. Tools

▪ Clarifications

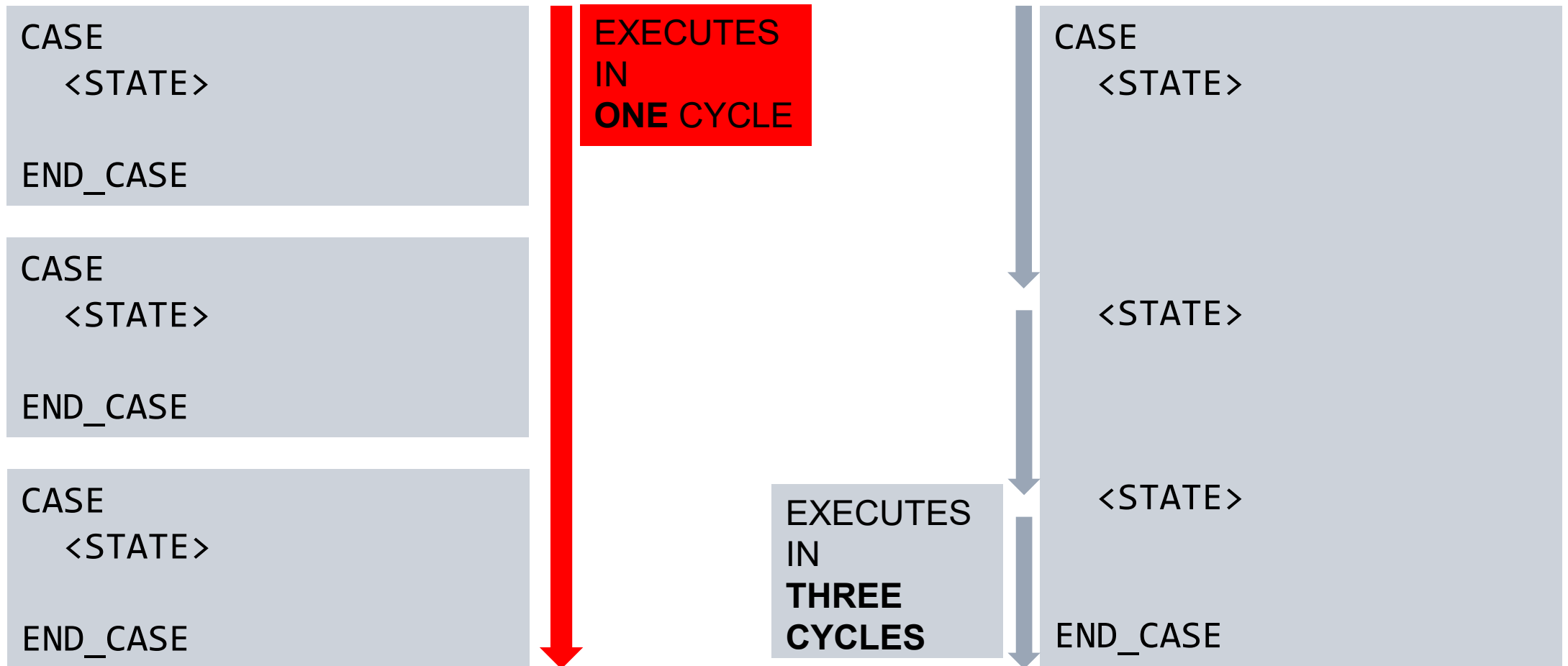
- The **ACID** tools require some explanations.
 - Saving data is one thing, saving machine state another.
- **Durability (design & physical constraint):** On the topic of a machine, it is vital that machine parts are not colliding in the physical world.
 - Fetch the data you need for recovery in time.
 - Use UPS for your IPC → save persistent data
 - Use the request for interruption ahead of any physical reaction from the machine to save the data
- Data after recovery is the last known logical state, **not** guaranteed physical reality.
 - Before motion is enabled after restart, recovery must evaluate data regarding safety status, drive/axis/mover state, position and reference validity, and process feedback.
- Design a recovery strategy for the machine
 - A generic solution is either not working or complex as hell.

3. Examples

- **Break the cycle between states with E_PROGRESS.**
 - Core feature of the required Same Cycle Responsiveness.
 - Ensure logical atomicity for bounded local state progression in the cyclic RTS.
 - Logical atomicity applies to local decisions and coherent command publication. Physical effects are asynchronous and cannot generally be rolled back.
- Ensure machine readability without having to construct machine semantics on top.
- Ensure human readability without translation of numbers into text.
- Time Optimal Programming while keeping source code readable.

3. Examples

- Break the cycle between states with E_PROGRESS.



3. Examples

- Break the cycle between states with E_PROGRESS.

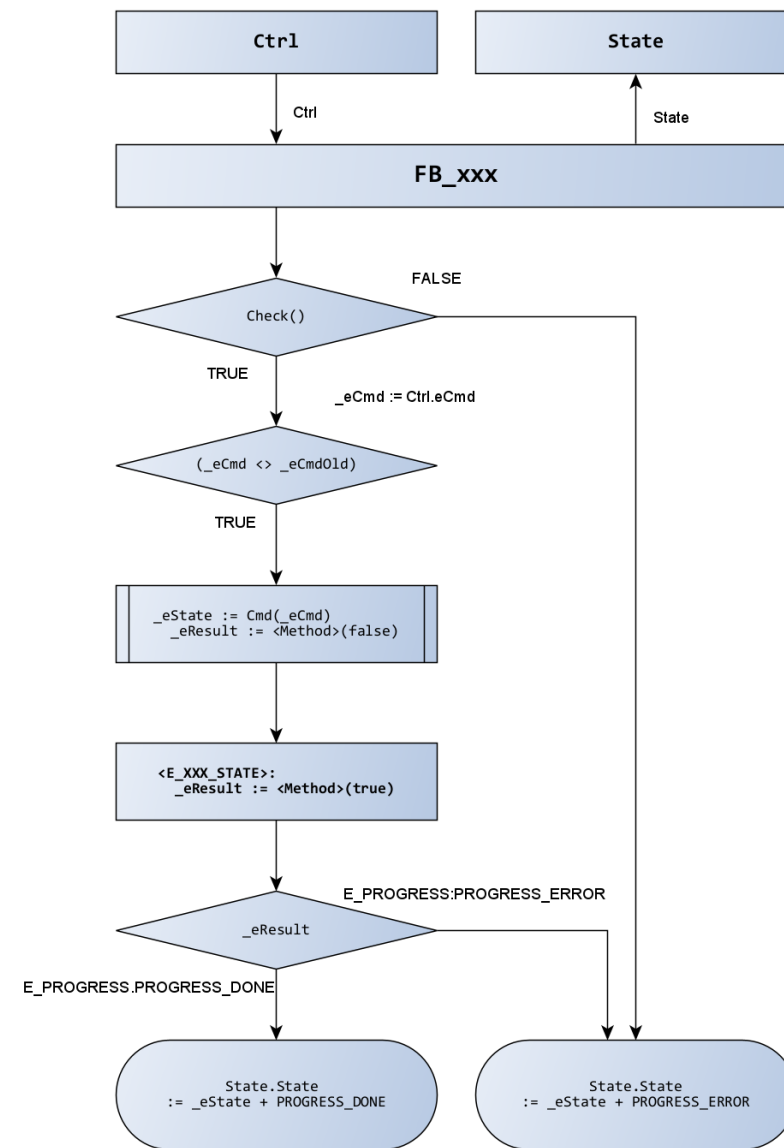
```
{attribute 'to_string'}
TYPE E_PROGRESS :
(
  // progress has 2 use cases
  // 1. as offset to cyclic interface's state
  //    for the requested command/function
  //    e.g. State := <Enum equivalent to eCmd> + E_PROGRESS
  //
  // 2. as state feedback (result) from (any) method
  PROGRESS_INVALID,
  PROGRESS_NOT_EXIST      := 100,
  PROGRESS_INIT           := 1000,
  PROGRESS_BUSY           := 2000,
  PROGRESS_PREPARE        := 3000,
  PROGRESS_STARTUP        := 4000,
  PROGRESS_CHECK           := 5000,
  PROGRESS_OCCUPIED       := 6000,
  PROGRESS_WORKING        := 7000,
  PROGRESS_STILL_WORKING  := 8000,
  PROGRESS_ERROR          := 9000,
  PROGRESS_DONE           := 10000
)UINT;
END_TYPE
```



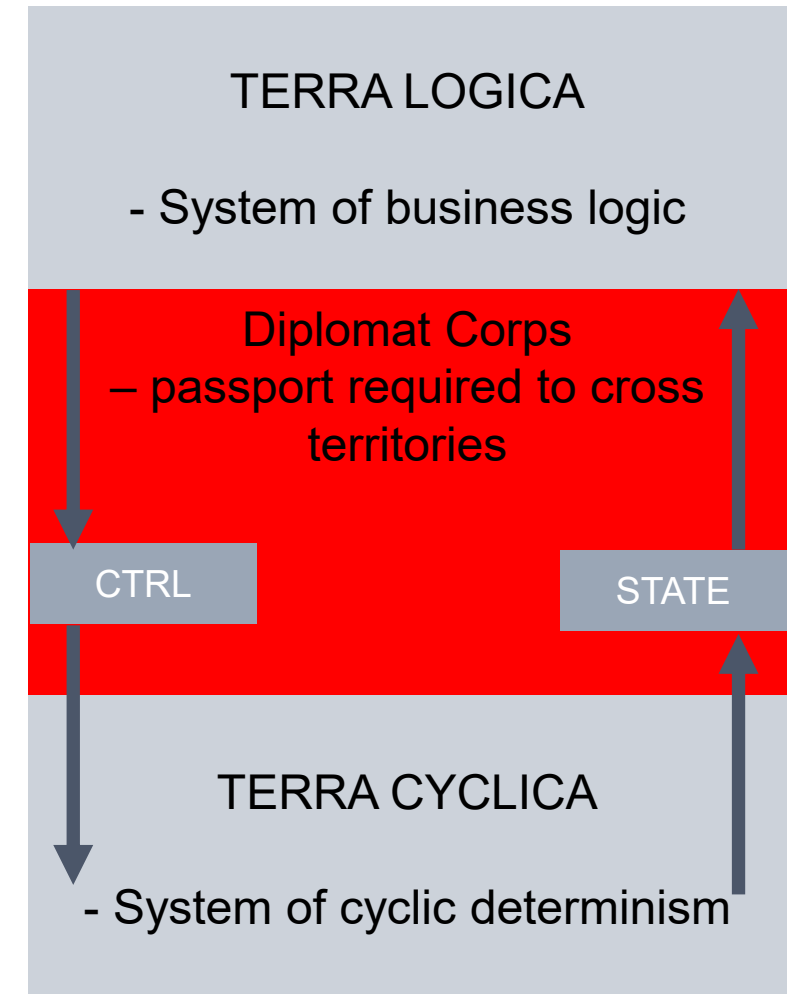
Shows
progress

3. Examples

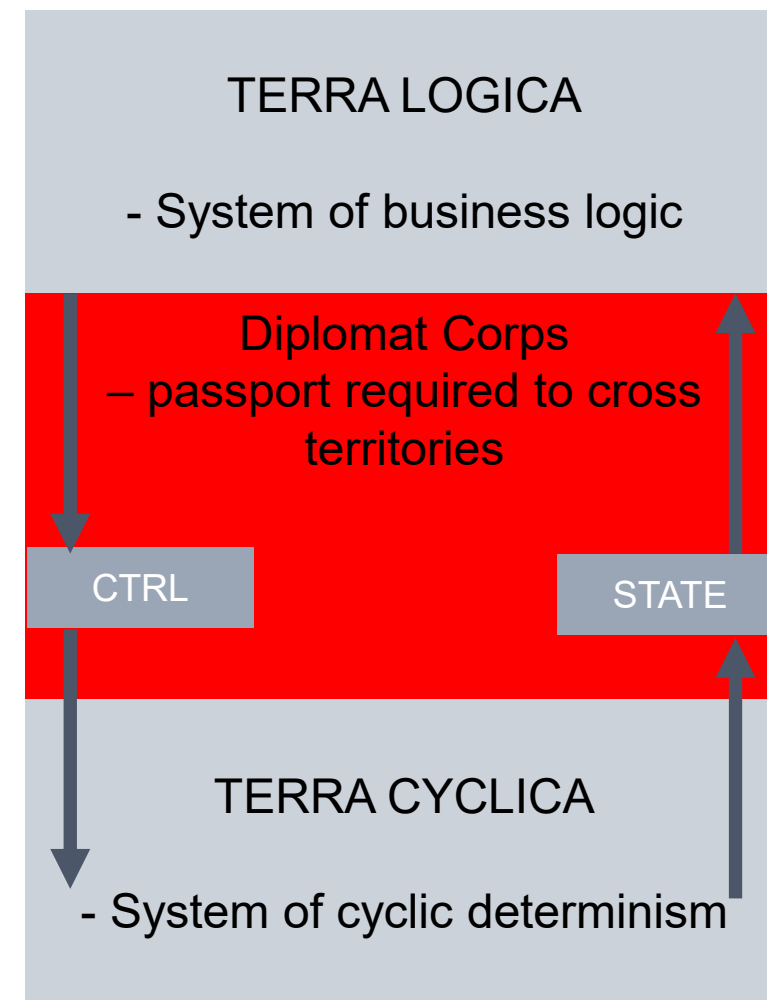
- **The Facade:**
 - State / Ctrl structures
 - Establishes unified access
 - Commands can simply be 'dropped' into Ctrl datafield by its owner
 - Enables asynchronous communication
 - Enables cyclic communication mapped onto any cyclic fieldbus
 - State is updated by own cyclic call
- State feedback for cyclic class wrappers
 - **Always** combined with E_PROGRESS
 - You can filter your response by a simple modulo division



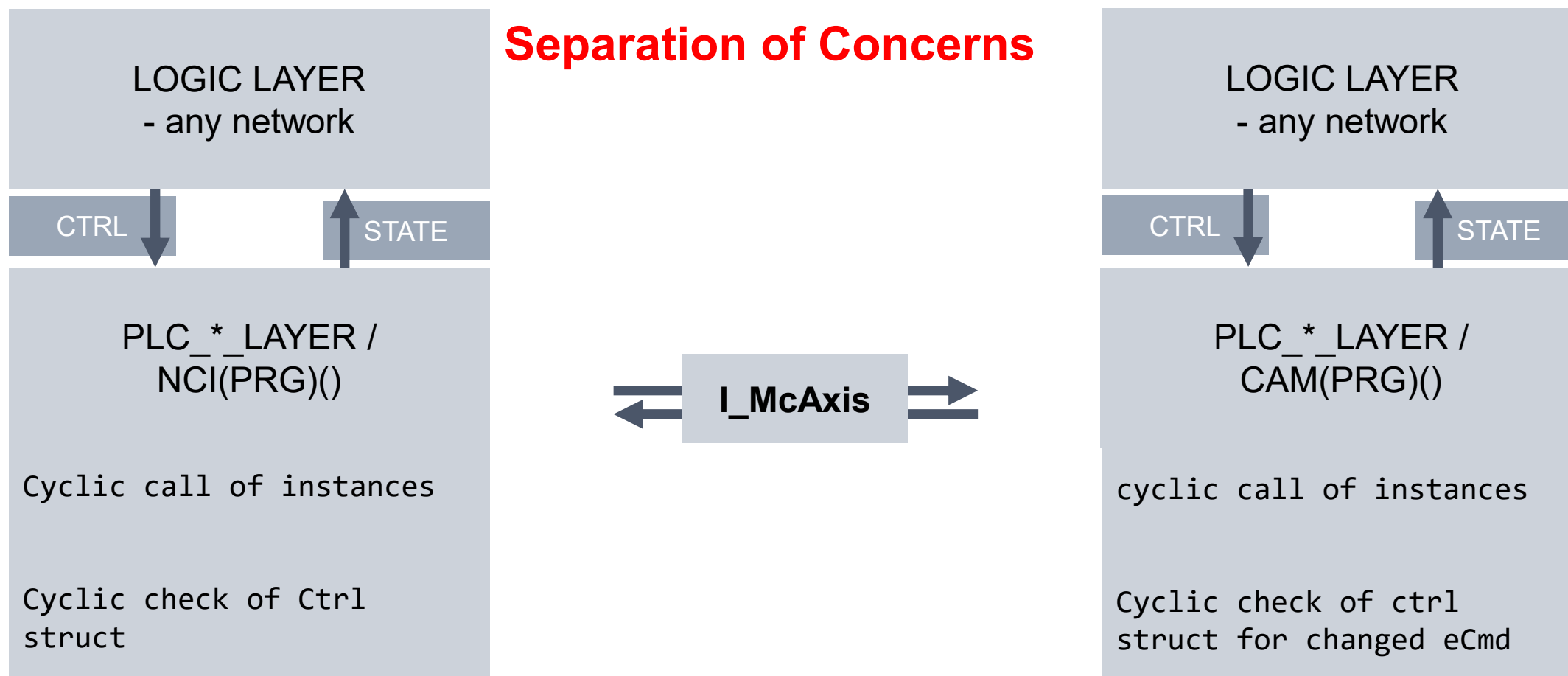
- **Diplomats and Territories - a tale of two systems**
- This world view depends on discipline not compiler restraints.
- The architecture operates on the principle of strict namespace encapsulation.
- The core TcGVL (Global Variable List) files are treated as sovereign territory.
- No external logic is permitted to read or write to this territory directly.



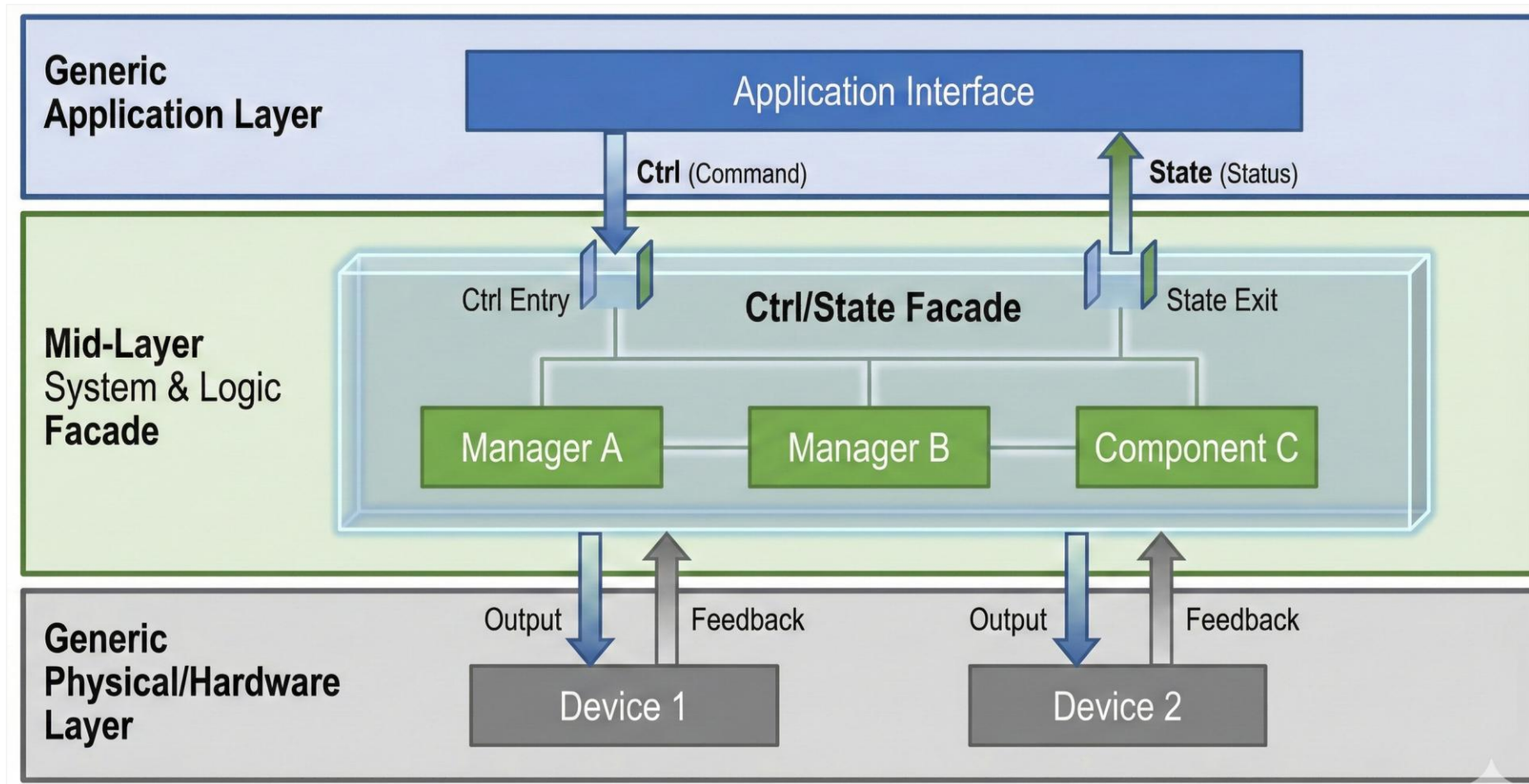
- **Diplomats and Territories - a tale of two systems**
- Instead, communication requires a "diplomat"—a strictly defined Facade or Interface.
- Any cyclic application may be connected via **MAPPING** procedures (fieldbus, EAP).
- Any non-cyclic application may be connected via symbolic access.
- Hybrid access possible
 - A middleware layer to organize access (switching between ADS and cyclic mapping) keeps the base layer unchanged



Software Design Example:



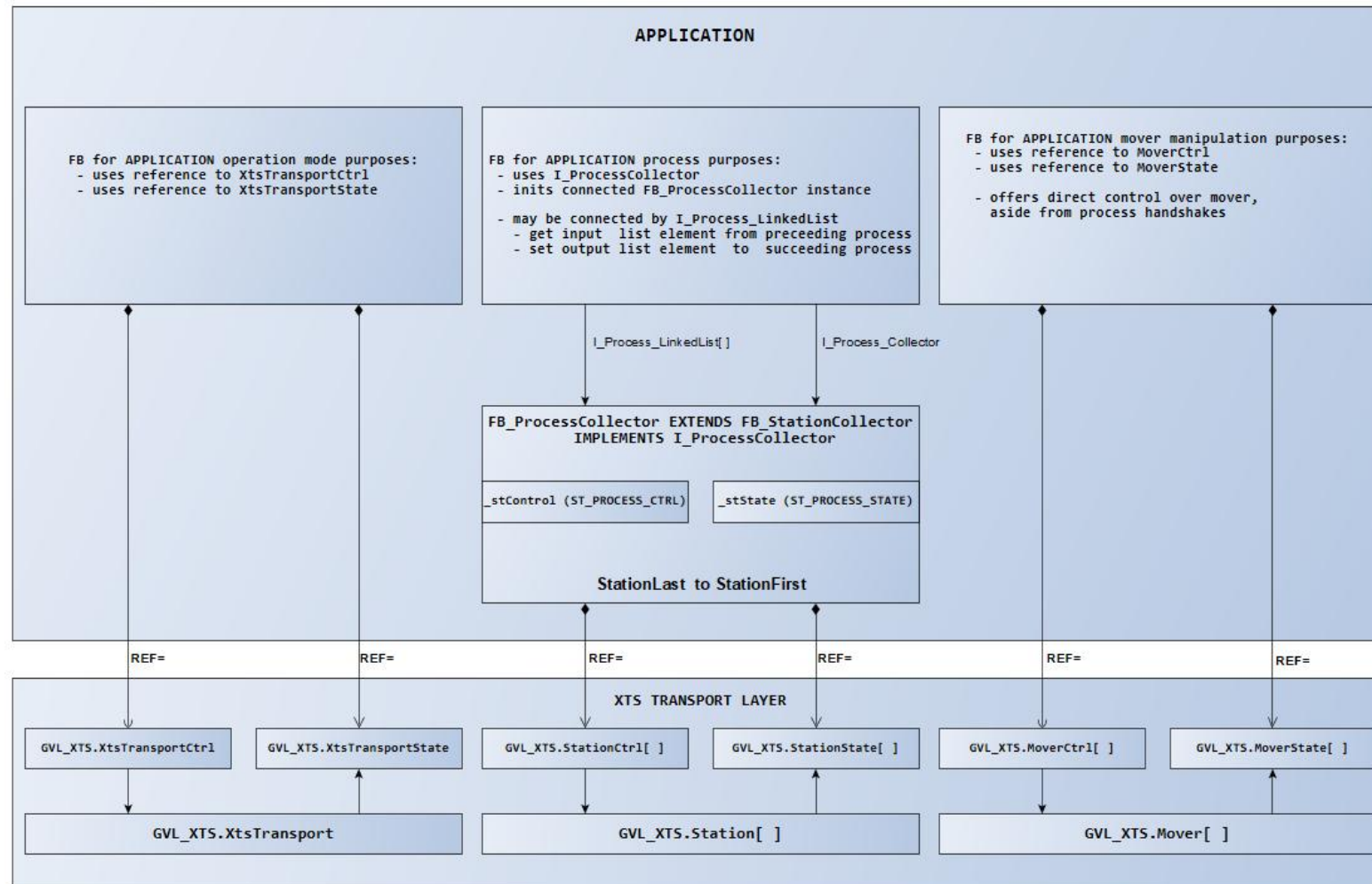
- Software Design Example: **The Facades**



■ Software Design Example:

- Build applications layer by layer
- The transport logic lives in instances

The Facades



- **Software Design Example:** **The Facades**
 - **MAIN(XtsTransport)**
 - MAIN — application orchestration and cyclic instance calls

```
135 // cyclic call
136 GVL_XTS.XtsTransport();
137
138 // main control
139 GVL_XTS.XtsTransport.Ctrl      REF= GVL_XTS.XtsTransportCtrl; // main control
140 GVL_XTS.XtsTransport.State    REF= GVL_XTS.XtsTransportState; // main state
141
142 // member controls
143 GVL_XTS.XtsTransport.XpuCtrl  REF= GVL_XTS.XpuCtrl;
144 GVL_XTS.XtsTransport.XpuState REF= GVL_XTS.XpuState;
145 GVL_XTS.XtsTransport.XpuInfo REF= GVL_XTS.XpuInfo;
146
147 GVL_XTS.XtsTransport.GroupItf := GVL_XTS.CaGroupItf;
148 GVL_XTS.XtsTransport.GroupInfo REF= GVL_XTS.CaGroupInfo;
149
150 GVL_XTS.XtsTransport.MoverItf REF= GVL_XTS.MoverItf;
151 GVL_XTS.XtsTransport.MoverInfo REF= GVL_XTS.MoverInfo;
152 GVL_XTS.XtsTransport.MoverLastPosition REF= GVL_XTS.LastPosition;
153 GVL_XTS.XtsTransport.MoverLastGap REF= GVL_XTS.LastGap;
154
155 GVL_XTS.XtsTransport.StationStartIndex := GVL_XTS.StationStartIndex;
156 GVL_XTS.XtsTransport.StationStart REF= GVL_XTS.StationStart; // start position for E_XTS_TRANSPORT_CTRL.CMD_TRANSPORT_START
157 GVL_XTS.XtsTransport.StationCtrlItf REF= GVL_XTS.StationCtrlItf; // station interfaces are required for accessing station methods
158 GVL_XTS.XtsTransport.StationListsItf REF= GVL_XTS.StationListItf; // list interface is required for access to linked list methods
159 GVL_XTS.XtsTransport.StationControl REF= GVL_XTS.StationCtrl;
160 GVL_XTS.XtsTransport.StationState REF= GVL_XTS.StationState;
```

■ Software Design Example:

The Facades

■ GVL_APPLICATION (ExternControl)

- GVL_APPLICATION — application-facing command facades and instance declarations.

```

GVL_APPLICATION  x MAIN_APP
1  VAR_GLOBAL CONSTANT
2      MAX_PROCESS      : E_INSTANCE := E_INSTANCE.SENDER_BUFFER_INFEED;
3  END_VAR
4  {attribute 'qualified_only'}
5  VAR_GLOBAL
6      //-----
7      // Application
8      //
9      // this is the part you will have to change and adapt into your programming
10     //-----
11     //-----
12     // fb_Application:
13     // - procedures for getting fb_Instance classes and XTS_TRANSPORT_LAYER to work
14     //
15     // - Cmd/state behaviour similar to sw mechanics in fb_TransportUnit
16     // - E_INSTANCE_CMD / E_INSTANCE_STATE + E_PROGRESS
17     //   INIT          / INIT          : initialize processes and XtsStations
18     //   DISABLE       / DISABLED      : RemoveALLAxis from CA Group, disable CA Group, disable all movers
19     //   ENABLE        / ENABLED       : AddALLAxis to CA Group, Enable CA Group, Enable all movers
20     //   WORK          / WORKING       : CMD_TRANSPORT_START, enable handshakes, start writing list entries to BUFFER_INFEED
21     //   PROCEED       / PROCEEDING    : CMD_TRANSPORT_RESTART, enable handshakes, continue writing list entries to BUFFER_INFEED
22     //   FINISH        / FINISH        : stop writing list entries to BUFFER_INFEED, handshake processes until outfeeds are empty
23     //   CLEAR         / CLEARED       : clear all instances of fb_Instance, clear all XtsStations
24     //-----
25     ApplicationMain    : fb_Application;
26     AppControl         : ST_APP_CTRL;
27     AppState          : ST_APP_STATE;
28
29     //-----
30     // fb_Instance control and state
31     // - structs are used in fb_Application as reference
32     //-----
33     InstanceCtrl       : ARRAY[1..MAX_PROCESS] OF ST_INSTANCE_CTRL;
34     InstanceState      : ARRAY[1..MAX_PROCESS] OF ST_INSTANCE_STATE;
35
36     TicketListControl  : ST_LIST_CTRL;
37     TicketListState    : ST_LIST_STATE;
38     TicketListDataInput : ST_LIST_NODE_DATA; // datafield for you to write(add) your data
39     TicketListDataOutput : ST_LIST_NODE_DATA; // datafield for you to read(get) your data
40 END_VAR
    
```

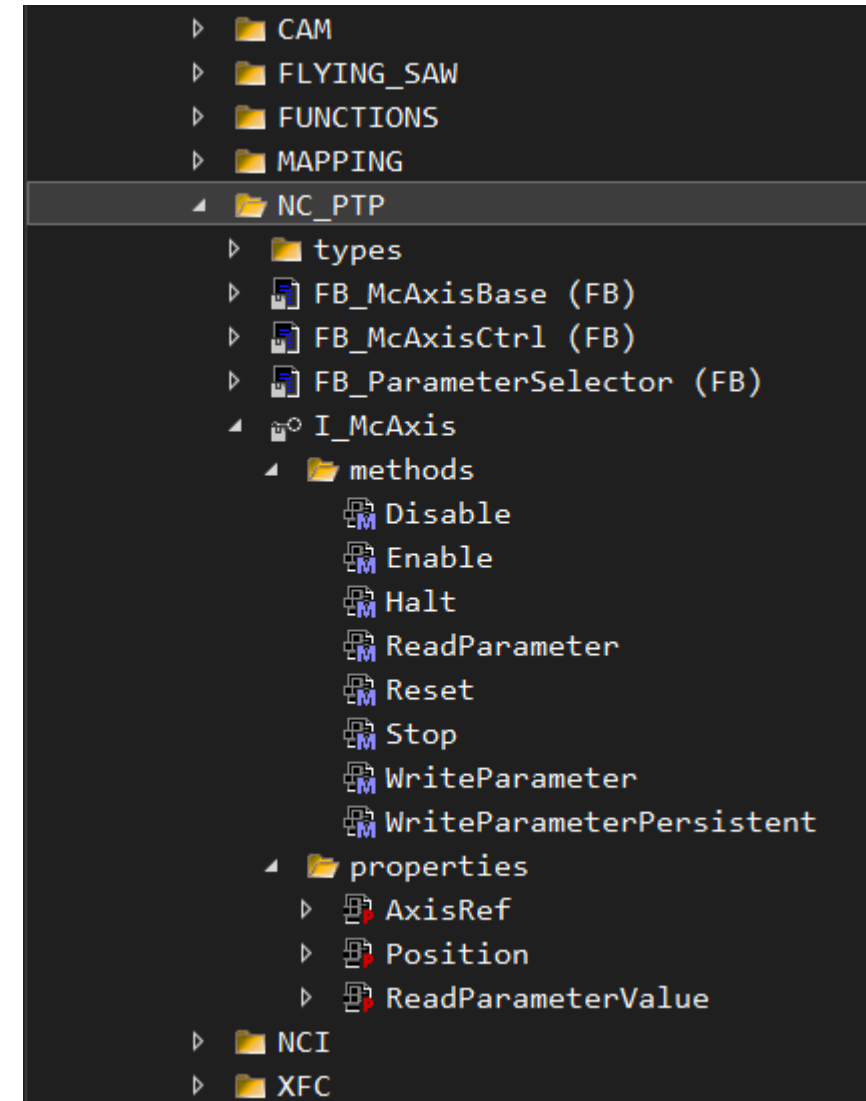
- **Software Design Example:** **The Facades**
 - **GVL_XTS(XtsTransport)**
 - GVL_XTS – transport-facing structures and instance declarations.

```
26
27 // ctrl / state datafields for extern control
28 StationCtrl      : ARRAY[1..MAX_STATION] OF ST_STATION_CTRL; // control for extern to write, you can do whatever you want as long as you do as I say
29 StationState     : ARRAY[1..MAX_STATION] OF ST_STATION_STATE; // state for extern to read, this is what I tell you to do
30
31 // station parameter
32 StationParameter : ARRAY[1..MAX_STATION] OF ST_STATION_PARAMETER; // wherever, whenever you want. Build it and they will come...
33
48
49 XtsTransportCtrl : ST_XTS_TRANSPORT_CTRL; // control of XTS transport for extern to write
50 XtsTransportState : ST_XTS_TRANSPORT_STATE; // state of XTS transport for extern to read
51
52 //-----
53 // MC2 encapsulation, CA movement
54 Mover      : ARRAY[1..MAX_MOVER] OF fb_MoverCtrl; // MC and CA methods for mover; cyclic check for individual command on mover
55
56 MoverCtrl   : ARRAY[1..MAX_MOVER] OF ST_MOVER_CTRL; // control of individual mover for extern to write
57 MoverState  : ARRAY[1..MAX_MOVER] OF ST_MOVER_STATE; // state of individual mover for extern to read
58
59 MoverSelector : fb_MoverSelector; // alternative for moving a single mover via Index/ID selection
60 MoverSelectorCtrl : ST_MOVER_SELECTOR_CTRL;
61 MoverSelectorState : ST_MOVER_SELECTOR_STATE;
```

■ Software Design Example:

- Distributes only required methods and properties.
- Enables strict separation of concerns and connects classes horizontally
- I_McAxis is the base interface and is not optional.

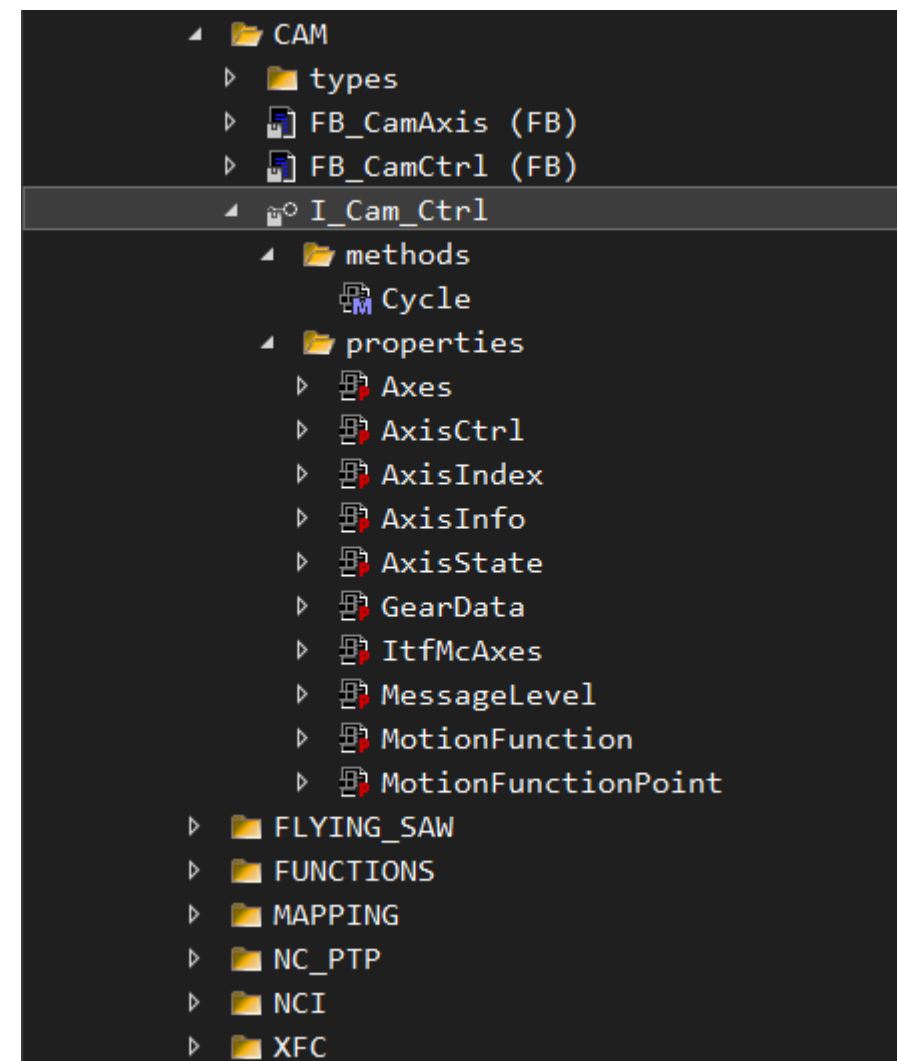
The Interface



■ Software Design Example:

The Interface

- Distributes only cycle method. The commands are written onto the facade structures.
- Enables use of empty interfaces for pre-compile defines
- Shall be void of any library datatypes from functions that are optional.



■ Software Design Example:

- The empty interface
- pre-compile defines
- Optional binding is possible with this
- The codebase does **not** need to change

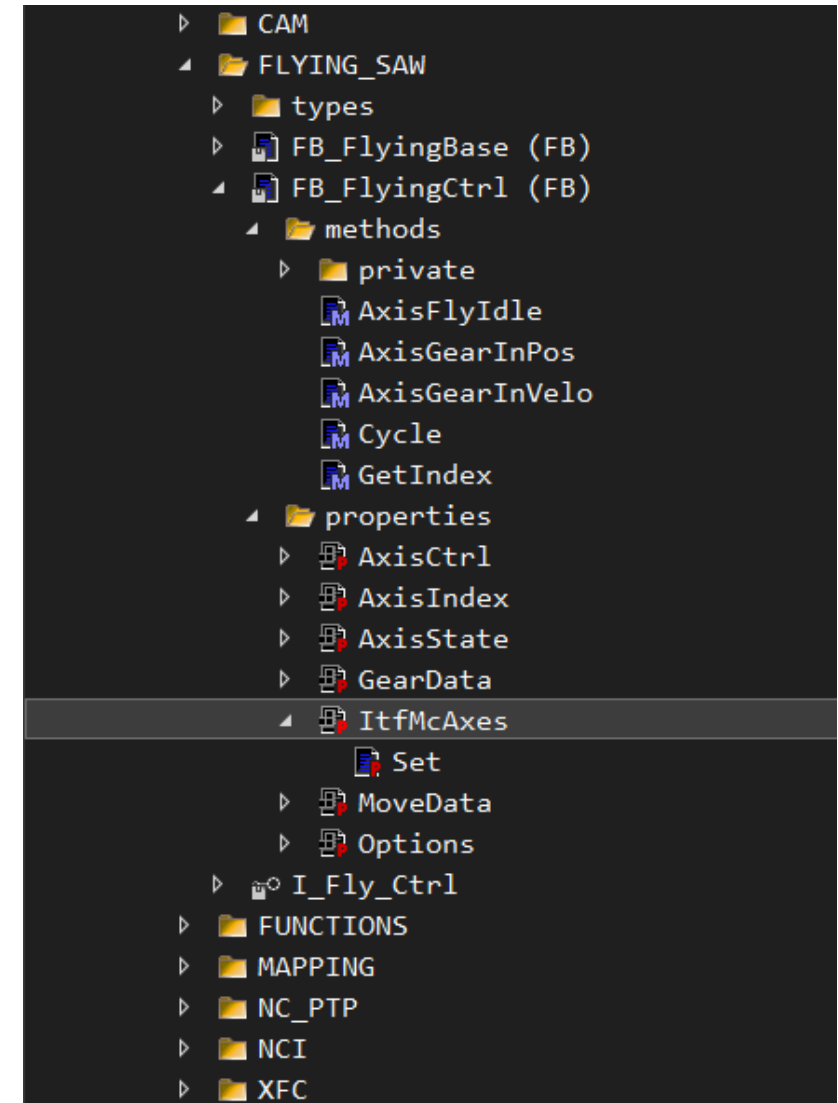
The Interface

```
//-----  
// cyclic interface function block: FB_CamCtrl  
// - MC_Camming functions are here  
//-----  
{IF defined (CAM)}  
  {info 'PLC_MOTION_LAYER: CAM'}  
  Control : ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF FB_CamCtrl;  
{ELSE}  
  Control : ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF I_Cam_Ctrl;  
{END_IF}  
END_VAR
```

■ Software Design Example:

- Injected in class via property
- Class is oblivious to implementation details
- This is the contract on offer

The Interface



- **Public on GitHub:**

- [https://github.com/haud-ba/PLC MOTION LAYER](https://github.com/haud-ba/PLC_MOTION_LAYER)
- [https://github.com/haud-ba/XTS TRANSPORT LAYER](https://github.com/haud-ba/XTS_TRANSPORT_LAYER)
- [https://github.com/HAUDX/XTS TRANSPORT EXAMPLES](https://github.com/HAUDX/XTS_TRANSPORT_EXAMPLES)